

Analysis of Algorithms

Dr. Satish Puri

Spring 2026

Let us introduce ourselves

What is an algorithm?

What is an algorithm?

- Problem solving
- A sequence of steps that solve a problem, within a finite amount of time, given a set of well-defined input.

Efficiency

- Problem size (Input parameters)
 - Run-time
 - Memory space
 - Data movement (from slow memory to fast memory)
- Output parameters like output size

Size of Input

- To make our count generally useable, we express it as a function of the size of the problem.
- Sequential Search: N numbers, order N time or $O(N)$
- Binary Search
searching a word in a dictionary
cut search area in half at each step
 $O(\log_2 N)$
If $N = 1024$ words,
#searches = $\log 1024 = \log (2^{10}) = 10$ steps

Measuring efficiency

- Why not run the programs and measure execution time to compare?
- Theoretical vs Experimental

Anecdote

- Boss vs Programmer

Fast FFT algorithm

- Detect nuclear explosion from seismographic data.
- Circa 1963, Fourier transform computation was slow – not practical.
- **Real-time Was Impossible:** For large datasets (like audio or seismic waves), computers would choke, making real-time analysis impossible.

The legend of the FFT algorithm (Fast Fourier Transform) is how James Cooley and John Tukey's 1965 algorithm transformed the slow Discrete Fourier Transform (DFT) from a theoretical tool to a practical powerhouse, cutting computation time from **$O(N^2)$ to $O(N \log N)$** using a "divide-and-conquer" strategy, enabling digital audio/video compression, medical imaging (MRI), and seismic analysis, with its origin linked to US efforts to detect Soviet nuclear tests.

Linear Programming (LP)

- **Cold War Era:** US's superior *application* of computing, fueled by LP and operations research, gave it a crucial edge in complex military planning, logistics, and technology, especially in the [Computer Arms Race](#), while the Soviets lagged, focusing more on theoretical math but failing to integrate it into effective strategic systems, contributing to their eventual technological and economic disadvantage. The US leveraged LP for resource allocation, missile defense, and command systems, creating a vast technological gap that central planning couldn't match
- **Simplex algorithm**
- **The "Good Will Hunting" Tale (A Related Legend):** A popular story (similar to *Good Will Hunting*) tells of a late-arriving Dantzig solving two famously difficult, unsolved statistics problems on a blackboard, mistaking them for homework, showcasing his innate ability

Comparing algorithms

- Experimental approach
 - Run the code
 - Measuring actual time
 - Problems with time
- Theoretical approach (asymptotic approach)
 - use a model
 - not accurate, approximate but useful
 - simplicity
 - order of growth in terms of input problem size

Counting Operations

- To measure the complexity of an algorithm we attempt to count the number of basic operations required to complete the algorithm.
- Rather than count all operations, select a fundamental operation, an operation that is performed “the most”, and count it.

Upper bound or worst case

- Big O

Time complexity using Big O notation

Searching One Array

Input: array A of n integers, and an integer t .

Output: Whether or not A contains t .

```
for  $i := 1$  to  $n$  do
    if  $A[i] = t$  then
        return TRUE
return FALSE
```

Time complexity using Big O notation

Searching One Array

Input: array A of n integers, and an integer t .

Output: Whether or not A contains t .

```
for  $i := 1$  to  $n$  do
  if  $A[i] = t$  then
    return TRUE
return FALSE
```

Quiz 2.1

What is the asymptotic running time of the code above for searching one array, as a function of the array length n ?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n^2)$

Searching Two Arrays

Input: arrays A and B of n integers each, and an integer t .

Output: Whether or not A or B contains t .

```
for  $i := 1$  to  $n$  do
  if  $A[i] = t$  then
    return TRUE
for  $i := 1$  to  $n$  do
  if  $B[i] = t$  then
    return TRUE
return FALSE
```

What, in big-O notation, is the running time of this longer piece of code?

Quiz 2.2

What is the asymptotic running time of the code above for searching two arrays, as a function of the array lengths n ?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n^2)$

Checking for a Common Element

Input: arrays A and B of n integers each.

Output: Whether or not there is an integer t contained in both A and B .

```
// outer loop
for  $i := 1$  to  $n$  do
  // inner loop
  for  $j := 1$  to  $n$  do
    if  $A[i] = B[j]$  then
      return TRUE
return FALSE
```

The question is the usual one: in big-O notation, what is the running time of this piece of code?

Quiz 2.3

What is the asymptotic running time of the code above for checking for a common element, as a function of the array lengths n ?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n^2)$

Checking for Duplicates

Input: array A of n integers.

Output: Whether or not A contains an integer more than once.

```
for  $i := 1$  to  $n$  do
  for  $j := i + 1$  to  $n$  do
    if  $A[i] = A[j]$  then
      return TRUE
return FALSE
```

Checking for Duplicates

Input: array A of n integers.

Output: Whether or not A contains an integer more than once.

```
for  $i := 1$  to  $n$  do
  for  $j := i + 1$  to  $n$  do
    if  $A[i] = A[j]$  then
      return TRUE
return FALSE
```

Quiz 2.4

What is the asymptotic running time of the code above for checking for duplicates, as a function of the array length n ?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n^2)$

Answer

Count how many times the if comparison happens.

For $i=1$, inner loop j repeats $n-1$ times

For $i=2$, inner loop j repeats $n-2$ times

For $i=3$, inner loop j repeats $n-3$ times

...

For $i=n-1$, inner loop j repeats 1 times

For $i=n$, inner loop j repeats 0 times

Checking for Duplicates

Input: array A of n integers.

Output: Whether or not A contains an integer more than once.

```
for  $i := 1$  to  $n$  do
  for  $j := i + 1$  to  $n$  do
    if  $A[i] = A[j]$  then
      return TRUE
return FALSE
```

Answer

Count how many times the if comparison happens.

For $i=1$, inner loop j repeats $n-1$ times

For $i=2$, inner loop j repeats $n-2$ times

For $i=3$, inner loop j repeats $n-3$ times

...

For $i=n-1$, inner loop j repeats 1 times

For $i=n$, inner loop j repeats 0 times

In total for different values of outer loop variable i , inner loop repeats

$= 1 + 2 + \dots + n-1 = \text{sum of consecutive integers from 1 to } n-1 = n(n-1)/2 = n^2 - n/2 = O(n^2)$

Checking for Duplicates

Input: array A of n integers.

Output: Whether or not A contains an integer more than once.

```
for  $i := 1$  to  $n$  do
  for  $j := i + 1$  to  $n$  do
    if  $A[i] = A[j]$  then
      return TRUE
return FALSE
```

Why Big O

Three Complexity Cases

- **Best case complexity** Related to the minimum number of steps required by an algorithm, given an ideal set of input values in terms of efficiency.
e.g., findMin (sorted array)
- **Average case complexity** Related to the average number of steps required by an algorithm, calculated across all possible sets of input values.
- **Worst case complexity** Related to the maximum number of steps required by an algorithm, given the worst possible set of input values in terms of efficiency.
e.g., findMin(reverse sorted array)

To simplify analysis yet still provide a useful approach, we usually use **worst case complexity**.

Different Cases

- Best case e.g. insertion sort is $O(N)$ for sorted input
- Worst case e.g. quicksort becomes $O(N^2)$
- Average case $O(N \log N)$

What are we going to learn

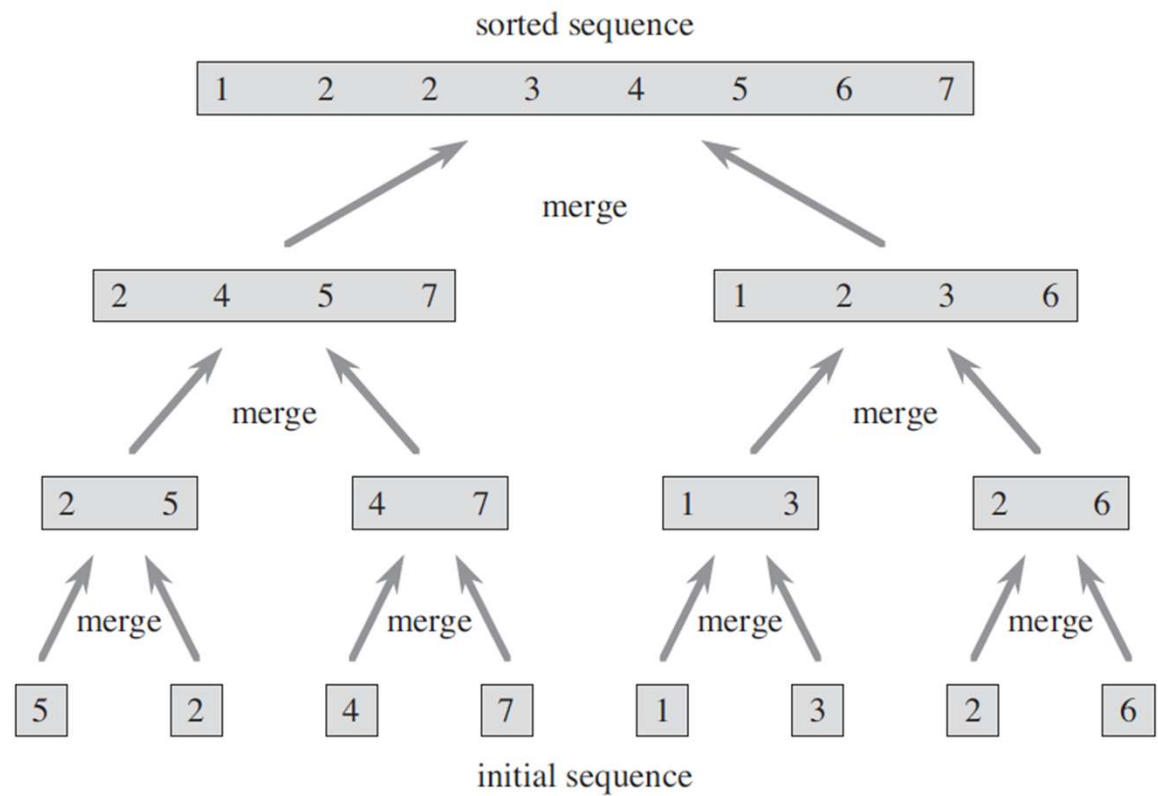
- Algorithmic Paradigms
- Efficiency Analysis using time and space complexity
- Correctness of an algorithm
- Fast methods
- Better programming and problem solving

Topics

Asymptotic Analysis	Parallel Algorithms
Divide and Conquer	Dynamic Programming
Greedy Algorithms	NP-Hard problems
Graph Algorithms	Approximation algorithms

Divide and Conquer

- Merge sort
- Quicksort



Divide and Conquer

- Merge sort
- Quick sort
- Does Selection sort or Insertion sort follow Divide and Conquer?

Data Structure

- Hashing, Cuckoo hashing
- Bloom filter
- Graphs

Graph Algorithms

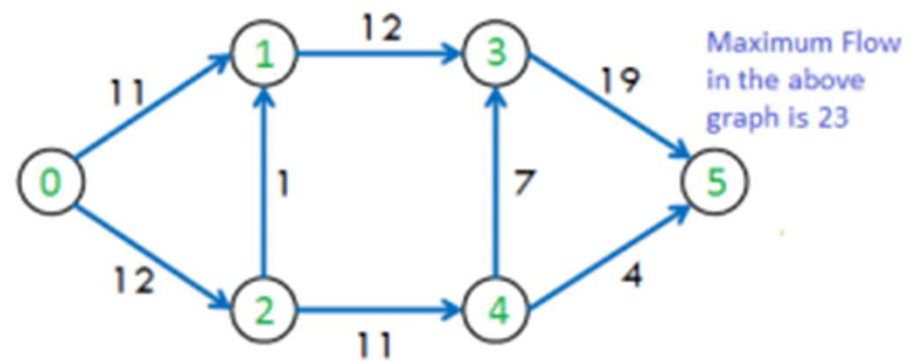
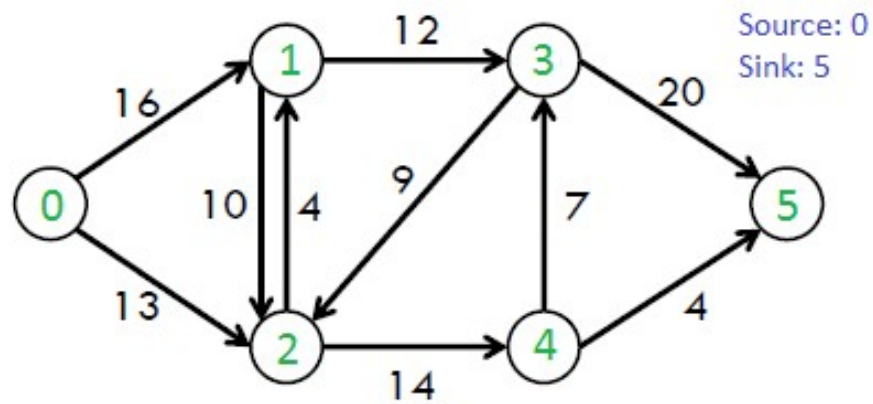


Figure. Max flow problem on a graph.
Capacity constraints are the edge weights. Nodes are labelled 0 to 5.
Max flow is 23 in this example.

Graph Algorithms

Shortest Path

Maximum Flow problem:

- a) Determine the maximum amount of "stuff" (data, oil, traffic) that can travel from a source to a sink in a network, subject to edge capacities.
- b) Separate foreground objects from the background in digital images
- c) Airlines use this algo. to maximize the time their fleet spends in the air (generating revenue) rather than on the ground.

Minimum Spanning tree

Greedy Algorithms

- Make the locally optimal choice at each stage
- Min Spanning Tree is greedy

Demo:

https://en.wikipedia.org/wiki/Prim%27s_algorithm

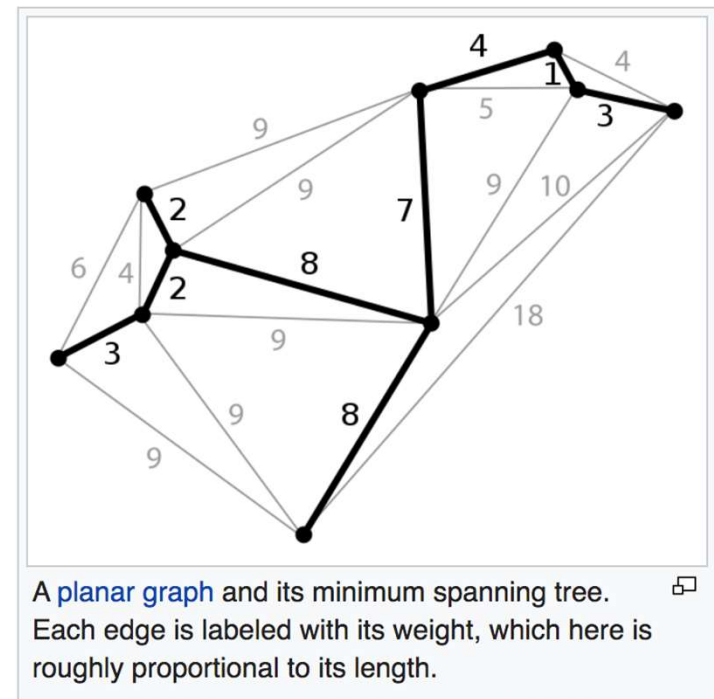


Figure. MST Graph and solution

Parallel Algorithms

- As opposed to Sequential Algorithm
- Multi-core CPUs and Manycore GPUs

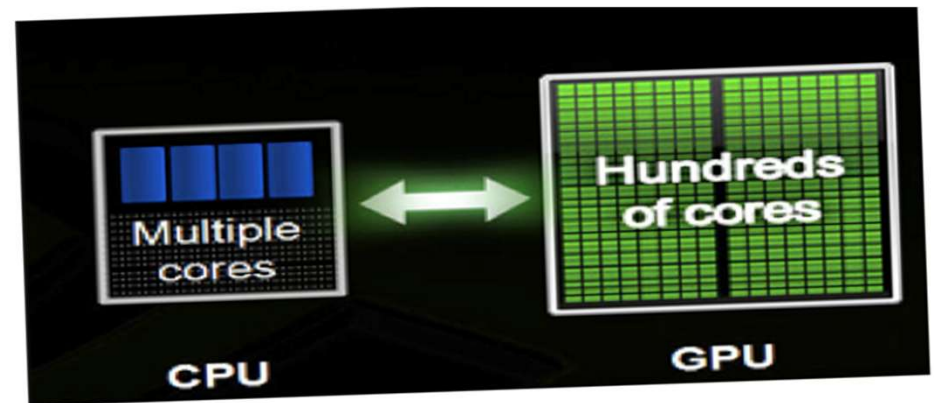


Figure. Parallel System

How difficult is a problem?

- Computational Complexity
- For simplicity, two classes: Easy vs Hard

Tractable vs Intractable problems

- How to know if a problem is hard or intractable?

Exponential rate of growth

According to the legend, the game of chess was invented by Sissa to amuse and teach his king. Asked by the grateful monarch what he wanted in return, the wise man requested that the king place on grain of wheat in the first square of the chessboard, two in the second, four in the third, and so on, doubling the amount of wheat up to the 64th square. The king agreed on the spot, and as a result he was the first person to learn the valuable – albeit humbling – lesson of exponential growth. Sissa's request amounted to $2^{64} - 1 = 18,446,744$ trillion grains of wheat, enough rice to pave all of india several times over.



Figure. By the time that the fifth square is reached on the chessboard, the board contains a total of 31

Million dollar for Algorithm discovery

- Clay Institute's Millennium Prize Problems – open questions
- P vs NP
- NP => Checking a solution easy, but finding a solution is difficult
 - Problems that have a good algorithm for checking a given possible solution but don't necessarily have a good algorithm for finding solutions are in the complexity class NP

e.g. Travelling Salesman Problem

Example of an intractable problem

- Integer factorization

15 is 5×3 and 70 is $2 \times 5 \times 7$.

Factorize a number with 200 digits

For a 10-digit number, we have billions of numbers we need to try out to see if they are factors. Number of possible factors increases exponentially with the number of digits.

- Internet security, e-commerce RSA
- Prime factorization is intractable for large primes
 - multiplying prime numbers is easy

<https://www.businessinsider.com/p-vs-np-millennium-prize-problems-2014-9>

Venn diagram on computational complexity

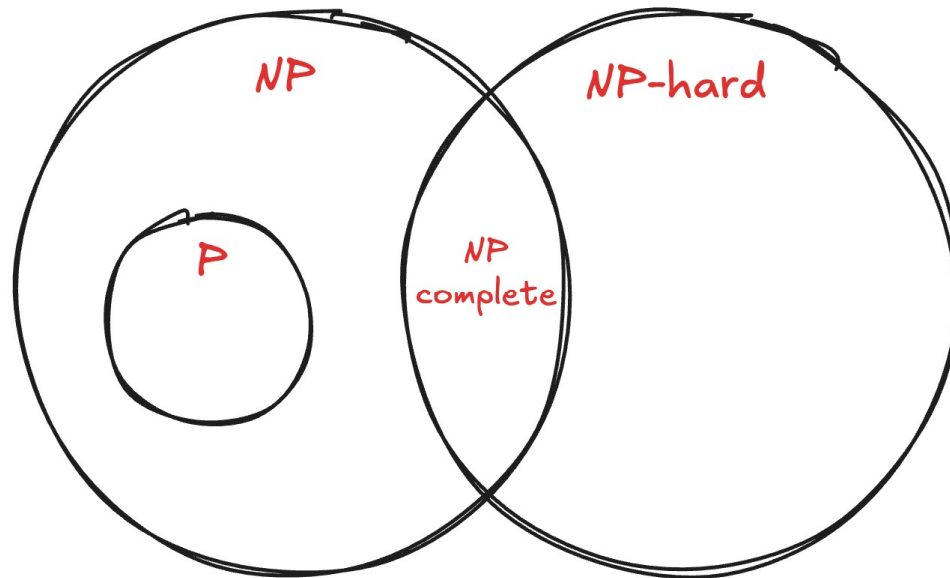


Figure. Venn diagram.

P is in *NP* because we can quickly check the solution. See *NP* definition.

NP-Complete problems

- Interesting Property
 - Finding a polynomial time algorithm for one of these problems would lead to a similarly quick algorithm for any NP problem. These problems are called [NP-complete](#).
 - You either need to find an efficient algorithm for one NP complete problem, or prove that no such algorithm exists for one particular such problem.

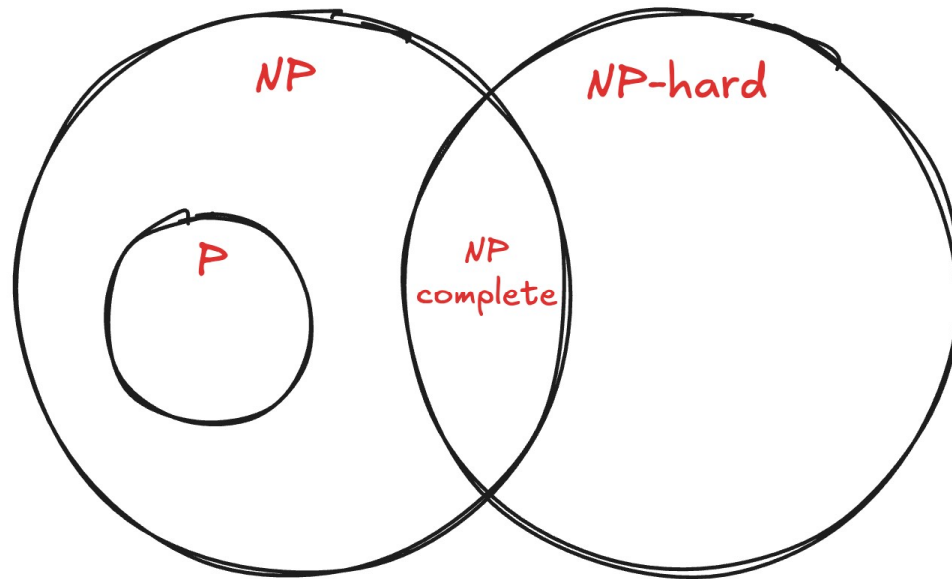


Figure. Venn Diagram

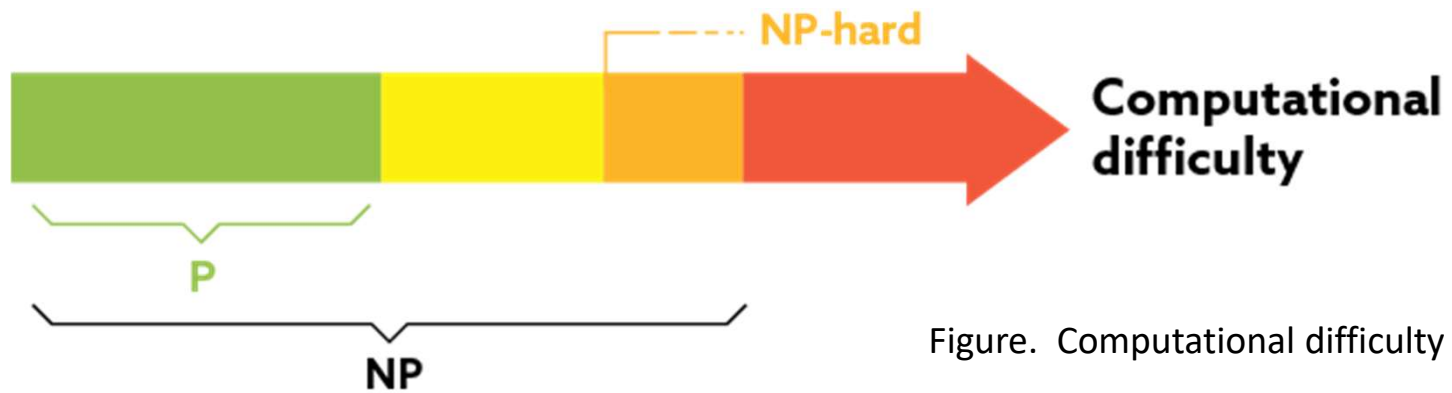


Figure. Computational difficulty

Pop Quiz

- An NP-Complete problem can not be solved by a computer.
(True/False)

Pop Quiz

- An NP-Complete problem can not be solved by a computer.
(True/False)

There are no polynomial time algorithms for NP complete problems.

But there is no formal proof of this yet, and any such proof would almost certainly involve radical new insights into the nature of computation and algorithms.

And win a million dollar prize

Approximation Algorithms

- NP Hard problems take too long.
- Approximation algorithms provide near-optimal solution with a guarantee on solution quality
- Approx. algo take polynomial time
- This technique does not guarantee the best solution.